

Numerical Methods 2026

Problems on optimization

1. You are given the function

$$f(x, y) = 20 + x^2 - 10 \cos(2\pi x) + y^2 - 10 \cos(2\pi y).$$

Can you guess/estimate the ‘landscape’ defined by this function?

You are required to find its minima (hopefully the global minimum). Use a gradient-based method of your choice to see where the minima are. You can start from different initial points on a uniform grid in the region defined by $x \in [-5, 5]$ and $y \in [-5, 5]$ on the 2D plane and note where the searches converge.

2. Generalize this to $N = 10$ dimensions.

$$f(x_1, \dots, x_{10}) = 100 + \sum_{i=1}^{10} [x_i^2 - \cos(2\pi x_i)].$$

3. You have a collection of particles which interact pair-wise. The interaction between the i th and the j th particles are given by the Lennard-Jones potential

$$V(r_{ij}) = 4\epsilon \left[\left(\frac{\sigma}{r_{ij}} \right)^{12} - \left(\frac{\sigma}{r_{ij}} \right)^6 \right].$$

Find the how a given number of particles (N) of this type with arrange themselves in 3D space to minimize the interaction energy. $N = 2, 3$ and 4 are easily seen from intuition and analytics. Still, find these numerically to check if you get the correct results. Also try $N = 5$ and 6 .

Spare some thought on how to generate the initial ‘structures’, and whether it is enough to start from a single initial structure.

Numerical Methods 2026

Problems on random number generators (1)

Prasenjit Sen

1. Consider the following two random number generators based on the linear congruence method.

$$X_{i+1} = aX_i + c \bmod m$$

(i) $m = 256$ $a = 137$ $c = 187$,

(ii) $m = 714,025$ $a = 1,366$ $c = 150,889$.

What is the period of each generator?

Use these to generate random numbers in a square of sides 10 cm. Show these points on a 2D plot for a visual examination of how they are distributed in the square.

2. Consider the following probability density function.

$$P(x) = \frac{2}{\pi} \sqrt{1-x^2} \quad 0 \leq x \leq 1.$$

Generate 100,000 or more points randomly in the unit square by using a uniform random number generator. Accept a point (x, y) if $y < P(x)$, reject otherwise.

Now divide the x -range $[0,1]$ in bins of small width (dx), count the number of points with x coordinate in each bin, plot these as a bar plot and check if this resembles the above function $P(x)$.

3. Use the appropriate generator from numpy to sample 10,000 or more random numbers from a normal distribution with mean 50.0 and variance 4. Test if you get the right distribution.

Numerical Methods 2026

Test 1, MSc Students

Total Questions: 15 Time: 1 hr.

Allocated marks for each correct answer, -1 for each incorrect answer.

You must choose all correct options and no incorrect options in cases where there may be more than one correct options. Else, you get -1.

Name:

Roll no.

SECTION

9 × 2

Q1. Which of the following is a mutable Python container?

- (a) Tuple
- (b) String
- ☒ (c) List
- (d) Integer

Q2. What is the output of the following code?

```
x = 5
y = 2
print(x // y)
```

- (a) 2.5
- ☒ (b) 2
- (c) 3
- (d) 2.0

Q3. What is the result of the expression `set([1, 1, 2, 3])`?

- (a) [1,1,2,3]
- ☒ (b) {1,1,2,3}
- (c) {1,2,3}
- (d) Error

Q4. What does `range(3)` produce when used in a for loop?

- (a) 1 2 3
- ☒ (b) 0 1 2
- (c) 0 1 2 3
- (d) Infinite loop

Q5. Which of the following are built-in objects (or types) in Python?

~~(a)~~ int

~~(b)~~ list

~~(c)~~ str

(d) type



X

Q6. What is the output of the following code?

```
A = [1, 2, 3]
```

```
B = A
```

```
B.append(4)
```

```
print(A)
```

~~(a)~~ [1,2,3]

(b) [1,2,3,4]

(c) Error

(d) None

X



Q7. Choose the correct statement(s).

(a) Numerical methods will always have both truncation and round-off errors.

(b) It is impossible to have numerical methods without truncation errors.

~~(c)~~ A set of linear algebraic equations can be solved numerically without any truncation errors.

~~(d)~~ Numerical differentiations are particularly prone to round-off errors.



Q8. Choose the correct statement(s).

(a) Accurate numerical calculations should be performed in single precision as this requires less memory.

(b) Double precision floating point operations give higher accuracy.

(c) Single precision and double precision computations would produce the same result.

(d) Python float uses 32-bit floating point numbers.

N/A (0)

Q9. What is the shape of the result?

```
import numpy as np
```

```
A = np.array([[1],[2],[3]])
```

```
B = np.array([10,20,30])
```

```
print((A + B).shape)
```

(a) (3,)

(b) (3,1)

(c) (3,3)

~~(d)~~ Error

X



SECTION B

Q10. Consider the following two codes. Chose all the correct statement(s).

(A)

```
A = [i for i in range(200000000)]

for i in range(200000000):
    A[i] = A[i] + 5
    A[i] = A[i] + 8
```

(B)

```
X = [i for i in range(200000000)]

for i in range(200000000):
    X[i] = X[i] + 5

for i in range(200000000):
    X[i] = X[i] + 8
```

- (a) (A) runs faster because it has a smaller number of operations.
- (b) (B) is faster because it uses cache more efficiently.
- (c) (A) is faster because it uses CPU registers more efficiently.
- ~~(d)~~ (B) is slower because it has to access X[i]'s from the main memory again for the second loop.

Q11. What is the output?

```
import numpy as np
A = np.array([1, 2, 3])
print(A * 2)
```

- ~~(a)~~ [2,4,6]
- (b) [1,4,9]
- (c) Error
- (d) [1,2,3,1,2,3]

Q12. In Python, functions are treated as *first-class objects*. Which of the following operations are possible due to this property?

- (a) Passing a function as an argument to another function
- (b) Assigning a function to another variable name
- (c) Returning a function from another function
- ~~(d)~~ Calling a function using its name.

Q13. Consider the following two Python class definitions:

```
\textbf{Code A:}
```

```
class A(object):
    def greet(self):
        return "Hello"
```

```
\textbf{Code B:}
```

```
class B:
    def greet(self):
        return "Hello"
```

Suppose objects of both classes are created and the `greet()` method is called. Which of the following statements is correct?

- (a) Class A will work, but Class B will raise an error because it does not inherit from object
- ☒ (b) Both classes behave identically because in Python 3 all classes implicitly inherit from object
- (c) Class B behaves differently because it does not have access to methods of object
- (d) Class A and Class B behave differently because explicit inheritance changes method resolution.

Q14. Suppose you have a hypothetical computer which stores floating point numbers in the decimal system. Also, it uses four digits for the mantissa and one digit for the exponent. The maximum round-off error between the two smallest numbers on this computer is (show how)

- (a) 0.5
- (b) 10^{-13}
- (c) 0.5×10^{-13}
- (d) 0.5×10^{-12} .

N/A (0)

Q15. Consider the following python code. Choose the correct output?

```
def f(x, L=[]):
    L.append(x)
    return L
```

```
print(f(1))
print(f(2))
print(f(3))
```

(a)

```
[1]
[2]
[3]
```

☒ (b)

```
[1]
[1, 2]
[1, 2, 3]
```

(c)

```
[1]
[2, 3]
[3]
```

(d) The code raises an error because lists cannot be used as default arguments.

Numerical Methods 2026

Test 2, MSc Students

April 23, 2026

Answers must be written legibly in simple, clear and correct English. You will be graded based on your written answers (what they mean), and not what you intended to say.

Q1. We are calculating thermal average of some physical quantity in a classical system using Metropolis sampling and Monte Carlo integration. The states of the system, assumed to be discrete, are numbered such that $p_1^* \geq p_2^* \geq \dots p_N^*$, p_i^* being the equilibrium probability of the state labelled i . Recall that the acceptance probabilities were defined as

$$A_{ji} = \min\left(\frac{p_j^*}{p_i^*}, 1\right) \quad | \quad A_{ik} = \left(\frac{p_i^*}{p_k^*}, 1\right)$$

We consider three states i, j and k with energies E_i, E_j and E_k .

Case I: $E_i > E_j > E_k$. Consider the acceptance probability of moves from the highest energy state to the lowest energy state k to i). Using the above acceptance probabilities, show that the moves in the state space form a Markov process.

Case II: $E_i > E_k > E_j$. Considering acceptance probability A_{ik} in this case, show that this apparently results in a non-Markovian process. What happens when you impose the detailed balance condition?

2 + 3

Q2. We have a collection of N particles interacting with each other through a pair-wise interaction potential that depends on their separation as $V(r_{ij})$ the functional form for which is known. It is learnt from experience, and some heuristic arguments, that the number of local minima in the structure space of the particles increases exponentially with N . Therefore, it is nearly an impossible task to exhaustively search out the lowest energy structure(s) by brute force for large N . So we decide to devise a more intelligent approach to locate it/these (no guarantee that the lowest energy structure cannot be degenerate).

(i) In a maximum of two sentences, explain the limitations of any gradient-based optimization methods in locating the lowest energy structure(s).

(ii) Come up with a proposal as to how we can do a global search of the lowest energy structure(s) combining Metropolis sampling in the structure space and gradient-based local search methods. Note that the structure space for this problem is continuous. This makes it somewhat different from the Ising spin system in practice, but in principle there is no difference in the way one conceives of the Metropolis moves. You may wish to think in terms of the energy landscape we had discussed in the class (if you were present).

Organize your proposal along the following points (not necessarily in this order).

(a) Starting structure; (b) How you generate new structures; (c) Acceptance/rejection probability; (d) Where does the gradient-based local search become relevant? Note that we are only interested in the minima of structures, and not the exact shape of the energy landscape; (e) How do you think the lowest energy structures will be identified? (f) Comment on whether you can claim with absolute certainty that the lowest energy structure(s) have been found.

$$A_{ji} = \min\left(\frac{p_j^*}{p_i^*}, 1\right)$$

Numerical Methods 2026

Test 2, PhD/IPhD students

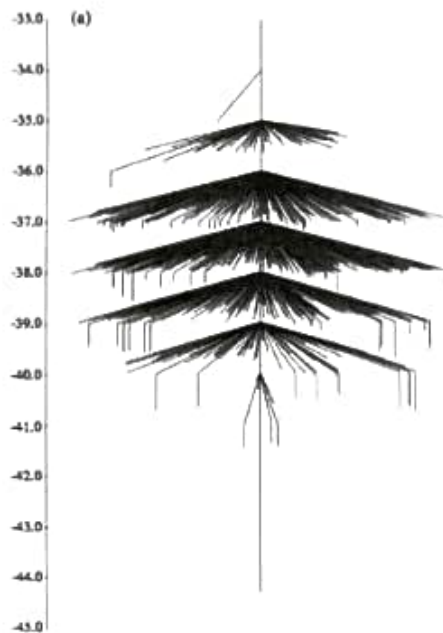
Coding problem

April 22, 2026

Consider a collection of 13 particles interacting via the Lennard-Jones potential

$$V(r_{ij}) = 4\epsilon\left[\left(\frac{\sigma}{r_{ij}}\right)^{12} - \left(\frac{\sigma}{r_{ij}}\right)^6\right].$$

The task is to use one or more gradient-based local optimization methods to locate the local minima in the energy landscape.



Start from at least 10 different initial structures (the more the better) and note the lowest energy structure you get in each case, along with its energy. You must clearly explain how you are generating different initial structures, and justify your procedure.

The known lowest energy structure has the energy of -44.3268 in units of ϵ .

Local energies of the LJ13 cluster are distributed as in the diagram. Compare your results with these.

Numerical Methods 2026

Test 2, PhD/PhD students

Coding problem

April 22, 2026

You have to do a Monte Carlo simulation of a 3D Ising model on a simple cubic lattice. Calculate the thermal average of energy and magnetization at $k_B T = 3J$.

The Ising model is described by the usual Hamiltonian

$$H = -J \sum_{\langle ij \rangle} \vec{S}_i \cdot \vec{S}_j.$$

Assume $J > 0$ and $S = 1$.

Plot energy and magnetization as functions of MC steps. Calculate the thermal averages and error bars.

Evaluate the susceptibility (per spin) from the following expression.

$$\chi = \frac{\beta}{N} (\langle M^2 \rangle - \langle M \rangle^2),$$

M being the total magnetization, and N is the number of spins.

Numerical Methods 2026

Test 2, PhD and IPhD Students

April 22, 2026

Answers must be written legibly in simple, clear and correct English. You will be graded based on your written answers (what they mean), and not what you intended to say.

Q1. We discussed the traveling salesman problem (TSP) in the class. Recall that in the canonical TSP, a salesman has to make a tour of N cities visiting every city only once. He starts from city 1 (which is fixed, let's say), visits every other city and gets back to the city of origin. The problem is to do so traveling the minimum possible distance.

(i) What is the computational complexity of the problem for an exhaustive search? If you have the right answer, you can easily check (after the exam) that even for a moderate value of $N = 20$, the problem becomes almost intractable by exact enumeration. In fact, it falls in the class of problems having the worst-case scaling.

(ii) Therefore, we decide to find the solution by simulated annealing. I hope some of you recall that annealing means taking the system to an elevated temperature to start with, allowing it to equilibrate, then cooling it *slowly* (annealing it), and stopping at some sufficiently low temperature (theoretically at $T = 0$).

Suggest how you would implement annealing for the TSP using Metropolis algorithm. In other words, what will be the 'moves', and write the elements of the acceptance probability matrix.

Mention clearly whether you are using (simple) Metropolis or generalized Metropolis in your proposal. Note, moves can be made in various ways. There are no right or wrong moves as long as the basic conditions of the problem statement are satisfied. Where they differ would be in their efficiency. But we are not worried about efficiency in this conceptual question. Moves you propose are your choice.

(iii) State with argument if your moves will give a Markov chain.

(iv) How would you ensure that the system is cooled slowly in a simulation?

(v) How do you think the solution would emerge from this process?

$$2 + 4 + 2 + 1 + 1$$

Q2. Consider the function

$$f(x, y) = 20 + x^2 - 10 \cos(2\pi x) + y^2 - 10 \cos(2\pi y).$$

Suppose you are using some conjugate gradient-based method to find the minima. You start from the initial point $(x, y) = (0.3, 0.0)$. What is the answer you will get?

Numerical Methods 2026

Test 2, PhD/IPhD students

Coding problem

April 22, 2026

Consider the following probability density function, which we have discussed.

$$P(x) = \frac{2}{\pi} \sqrt{1-x^2} \quad 0 \leq x \leq 1.$$

Use the rejection method to generate random numbers from this probability density function. You must have at least 50,000 numbers in the sequence.

Show by drawing a histogram that the chosen numbers actually follow the above distribution.

Show that $P(x)$ is not normalized.

Show that once we account for acceptance probability properly, the marginal probability distribution of the *accepted* x values is normalized.

$$\text{Norm}_P(n) = \frac{P(n)}{\text{Norm}} = \frac{4}{\pi} \sqrt{1-x^2}$$

Numerical Methods

Problem Set: 1

(I)

Given:

$$x = 5, \quad y = 5.0, \quad z = "5"$$

1. Print the type of each variable.
2. Convert z to an integer and store it in a new variable.
3. Check whether $x = y$.

(II)

Let:

$$s = "NumericalMethods"$$

1. Print the first 9 characters.
2. Print the last 7 characters.
3. Concatenate "2026" to s .
4. Convert the string to lowercase.

(III)

Given:

$$L = [3, 4, 7, 12, 13]$$

1. Add 20 at the end.
2. Replace 7 by 9.
3. Print the sum of all elements.
4. Reverse the list.

(IV)

Define the matrix:

$$A = \begin{bmatrix} 10 & 12 \\ 31 & 14 \end{bmatrix}$$

1. Print the element 31.
2. Change the element 12 to 16.
3. Print the first row.

(V)

Given:

$$T = (15, 30, 45)$$

1. Try to change the second element. What happens?
2. Convert the tuple to a list.

(VI)

Given:

$$S = \{1, 2, 3, 3, 3, 4, 4\}$$

1. Print S . What do you observe?
2. Add 20 to the set.
3. Remove 2 from the set.

(VII)

Given:

```
student = {"name": "Anik", "marks": 85}
```

1. Add key "course" with value "Numerical Methods".
2. Update marks to 93.
3. Print all keys and values.

(VIII)

Using NumPy:

1. Create an array of integers from 0 to 20.

2. Create a 3×3 matrix of zeros.
3. Create a 2×2 identity matrix.

(IX)

Given:

$$A = \text{np.array}([2, 5, 13, 43])$$

Compute:

1. $2A$.
2. A^2 .
3. The mean of A .

(X)

Given:

$$A = \begin{bmatrix} 4 & 7 \\ 5 & 8 \end{bmatrix}, \quad B = \begin{bmatrix} 3 & 0 \\ 1 & 3 \end{bmatrix}$$

Compute:

1. $A + B$.
2. AB .
3. The transpose of A .

(XI)

1. Create a python list containing numbers from 1 to 10.
2. Create a NumPy array containing numbers from 1 to 10.

Multiply both by 2 and compare the outputs.

(XII)

Create a list representing the vector

$$v = (1, 5, 9, 12, 21)$$

1. Multiply every element by 4.
2. Find the maximum and minimum values.

(XIII)

Let:

$$A = \{1, 2, 3, 4\}, \quad B = \{3, 4, 5, 6\}$$

Compute:

1. $A \cup B$
2. $A \cap B$
3. $A - B$

(XIV)

Using NumPy:

1. Create an array of 20 equally spaced numbers between 0 and 1.
2. Reshape it into a 4×5 matrix.

(XV)

Given:

$$A = \text{np.array}([1, 2, 3]), \quad B = \text{np.array}([4, 5, 6])$$

Compute:

1. $A + B$.
2. $A * B$.
3. $A \cdot B$.

(XVI)

Create a NumPy array $x = [2, 33, 14, 55]$ and compute $y = 3x^2 + 2x + 1$ without using loops.

Numerical Methods

Problem Set: 2

(I)

1. Generate 1000 random numbers uniformly distributed in $[0, 1]$. Compute their mean and variance.
2. Generate 10000 uniform random numbers in $[0, 1]$ and plot their histogram.
3. Generate 10000 gaussian random numbers with mean 0 and standard deviation 1. Plot their histogram and compare its shape with the uniform case.
4. Use `np.random.randint()` to simulate rolling a fair six sided die 100000 times. Compute the probability of getting each face.
5. Generate two independent uniform random variables p and q in $[0, 1]$. Compute and plot the distribution of $w = p + q$. What shape do you observe?
6. Generate 10000 gaussian random numbers and compute their cumulative sum. Plot the result and describe what kind of motion it resembles.
7. Use random numbers to estimate the value of the integral

$$\int_2^5 e^{-x} dx.$$

(II)

1. Write a program using a for loop that prints all numbers from 1 to 10.
2. Using a while loop, keep taking numbers from the user and add them. Stop when the user enters 0. Print the final sum.
3. Using a for loop, read 20 numbers and find the largest one (without using `max()`).
4. Using a while loop, keep adding natural numbers starting from 1:

$$1 + 2 + 3 + \dots$$

Stop when the sum becomes greater than 100. Print:

- The final sum.
- How many numbers were added.

III

1. Write a python function that takes a number as input from the user and returns:

- "Even" if the number is even,
- "Odd" if the number is odd,
- "Zero" if the number is 0.

Then call the function and print the result.

IV

Consider the function:

$$f(x) = e^{-t^2} \sin(5t)$$

defined on $t \in [0, 5]$, with N equally spaced points.

1. Compute the Discrete Fourier Transform (DFT) using the formula:

$$F[k] = \sum_{n=0}^{N-1} f[n] e^{-2\pi i kn/N}, \quad k = 0, 1, \dots, N-1.$$

2. Plot the magnitude spectrum $|F[k]|$.
3. Measure the computation time for this.
4. Compute the DFT efficiently using `numpy.fft.fft()`.
5. Plot the magnitude spectrum $|F[k]|$.
6. Measure the computation time.
7. Compare the computation times of DFT formula and numpy FFT.

Problem Set 3: Functions in python

Course: Numerical Methods

February 9, 2026

Instructions

- Use DRY principle for writing programs.
 - Read NumPy / SciPy documentation where required.
 - Avoid using external libraries unless stated.
-

Problem 1 — Basic Functions and Return Values

Write a function called `linspace` that takes two positional arguments: `a`, `b`, and one keyword argument `n`. It should generate equally spaced numbers between `a` and `b` and compute:

- sum of numbers
- mean of numbers
- difference between last and first value

Return all three values.

Then write another function that calls this function twice with different inputs and returns those values.

Problem 2 — Interpolation Using SciPy

You are given a sparse dataset representing measurements of a physical observable sampled at non-uniform points (for example, detector response or a thermodynamic quantity as a function of temperature).

Dataset

```
import numpy as np

x_data = np.array([0.0, 0.4, 1.1, 1.8, 2.5, 3.7, 4.2, 5.0, 6.3, 7.1])
y_data = np.array([0.02, 0.15, 0.83, 0.97, 0.61, -0.12, -0.45, -0.96,
                  -0.30, 0.20])
```

The points are irregularly spaced. Your goal is to construct smooth interpolating functions and evaluate them on a dense grid.

Part 1 — Configurable Interpolator

Write a function

```
def build_interpolator(x, y, *, kind="cubic"):
    ...
```

Requirements:

- Takes arrays `x` and `y`. (Discover the meaning of `*` in the arguments; <https://realpython.com/python-asterisk-and-slash-special-parameters/>).
- Takes one keyword-only argument `kind` that controls interpolation type.
- Uses a SciPy interpolation routine (discover the correct function and its arguments from the SciPy documentation).
- Supports at least:
 - linear
 - quadratic
 - cubic
- Returns a callable interpolating function.

Part 2 — Dense Grid Evaluation

The interpolator returned in Part 1 can evaluate values at arbitrary points. In this part, you will write a helper function that constructs a *dense, uniform grid* of points in a given interval and evaluates the interpolating function on that grid.

Write the function:

```
def evaluate_on_dense_grid(f, x_min, x_max, n=500):
    ...
```

Inputs:

- `f`: the callable interpolating function returned by `build_interpolator`.
- `x_min`: left endpoint of the interval.
- `x_max`: right endpoint of the interval.
- `n`: number of equally spaced grid points to generate (default = 500).

What your function must do:

1. Create a uniformly spaced array of `n` points between `x_min` and `x_max` (inclusive).
2. Call the interpolator `f` on this array to compute interpolated values.
3. Do not use loops — the interpolator should accept a NumPy array directly.

Return values:

- the dense grid array of x-values
- the corresponding array of interpolated y-values

Example expected behavior:

```
xg, yg = evaluate_on_dense_grid(f, 0.0, 7.0, n=1000)
```

After this call:

- `xg` contains 1000 evenly spaced points in $[0, 7]$
- `yg` contains the interpolated values at those points

This dense grid will be used in the next part for plotting smooth curves.

Part 3 — Visualization(Matplotlib)

- Build linear, quadratic, and cubic interpolators.
- Evaluate each on a dense grid.
- Plot original data points and all interpolated curves.
- Add legend and axis labels.

Problem 3 — Scope and Bytecode Inspection

Consider the following program:

```
x = 10

def outer():
    x = 20
    def inner():
        return x
    return inner
```

Tasks:

- Disassemble `outer` using the `dis` module
- Disassemble the returned function object
- Identify where Python loads the variable `x` from
- Modify the program so that `inner()` returns 10 without changing the return line

You must explain your answer using bytecode instructions.

Problem Set 4 (Numerical Methods)

Code (Problem 1.1 and Problem 1.2)

```
import numpy as np

def transform_operator(func):
    def wrapper(self, operator):
        operator = np.array(operator, dtype=complex)
        is_hermitian = np.allclose(operator, operator.conj().T)
        if not is_hermitian:
            operator = 0.5*(operator+operator.conj().T)
        return func(self, operator)
    return wrapper

class QM_objects:

    tag_line = "I follow principles of quantum mechanics!"

    def __init__(self, rest_mass, state):
        self.rm = rest_mass
        self.state = np.array(state, dtype=complex)

    def norm(self):
        nrm = np.vdot(self.state, self.state)
        return nrm

    def normalize(self):
        return self.state/np.sqrt(self.norm())

    def is_normalized(self, tol=1e-10):
        return np.isclose(self.norm(), 1, tol)

    @transform_operator
    def operate(self, operator):
        return operator @ self.state

    @transform_operator
    def expectation(self, operator):
        return np.vdot(self.state, self.operate(operator))

class Fermions(QM_objects):

    def stats(self):
        return "I follow FD statistics!"
```

Problem 1.1: Understanding Decorators

1. Explain in your own words:
 - (a) Why is the function `wrapper` defined inside `transform_operator`? What would break if we removed this nesting?
 - (b) What role does `func` play?
2. Modify the decorator such that:
 - (a) It prints a warning whenever a non-Hermitian operator is corrected.
 - (b) It keeps track of how many such corrections were made.(Hint: One way to do it is using global variable named, say, "count".)
3. Write a test script:
 - (a) Create one Hermitian and one non-Hermitian operator.
 - (b) Apply both using the `operate` method and demonstrate that your modification works correctly.
4. Why is it better to implement this logic using a decorator rather than duplicating the same code inside multiple methods?

Problem 1.2: Classes and Inheritance

1. Modify the `normalize()` method such that:
 - (a) It updates `self.state` in place (currently it doesn't).
 - (b) Calling it multiple times does not change the result after the first normalization.
2. Add a method:

```
def apply_sequence(self, operators):
```

This method should:

- (a) Take a list of operators.
- (b) Apply them sequentially to the state (from left to right).
- (c) Return the final transformed state.

Hints:

- Start with the current state stored in `self.state`.
- Use a loop over the list `operators`.
- At each step, apply the operator using matrix multiplication.
- Be careful: do you want to modify `self.state`, or work on a temporary variable?
- Think about code reuse: can you use the already defined `operate()` method?

- What happens if one of the operators is not Hermitian? (Your decorator should automatically handle this.)
3. Create a subclass `Bosons`:
 - (a) It should inherit from `QM_objects`.
 - (b) Add a method `stats()` returning an appropriate string.
 4. Write a test script that:
 - (a) Creates both fermion and boson objects.
 - (b) Demonstrates normalization.
 - (c) Shows that invalid input is properly handled.
 5. What is the advantage of using inheritance here instead of writing completely separate classes?

Problem 2:

Find the solutions of the following system of nonlinear equations using the `scipy` package. Use a `lambda` function to define the system:

$$\begin{cases} x^2 + y^2 = 1, \\ e^x + y = 1. \end{cases}$$

Problem 3:

Evaluate the following definite integral numerically using the `scipy.integrate` module:

$$I = \int_0^2 \frac{\sin(x^2)}{1+x^4} dx.$$

1. Define the integrand using a `lambda` function.
2. Use `scipy.integrate.quad` to compute the integral.
3. Print both the computed value of the integral and the estimated error.

Problem 4:

The following data represents measurements of a function $f(x)$:

x	0	1	2	3	4	5
$f(x)$	1.0	2.2	1.9	4.5	2.2	2.6

1. Use the `scipy.interpolate.CubicSpline` class to construct a cubic spline interpolation of the data.
2. Estimate the value of $f(2.5)$ using the spline.

3. Calculate and display the coefficients of the cubic polynomials that define the spline on each subinterval.
4. Plot the original data points and the cubic spline interpolation curve.

Numerical Methods 2026

Problems on matrices (1)

1. Solve the equations numerically using any matrix method.

$$2x_1 - 2x_2 + 4x_3 = 8$$

$$2x_1 - 3x_2 + 2x_3 = 8$$

$$x_1 - x_2 + x_3 = -3.5$$

2. Solve the following equations following these steps. (i) Form the matrix A if the equations are written as $A \cdot x = b$, (ii) use a routine to find A^{-1} , (iii) evaluate $x = A^{-1} \cdot b$.

$$x_1 + 2x_2 - x_3 = 3$$

$$3x_1 - x_2 + 2x_3 = 1$$

$$2x_1 - 2x_2 + 3x_3 = 2$$

3. Consider a 2D $N \times N$ (try to start from a small value of N , say 4 for easy visualization and try to do at least up to $N = 20$) square lattice. Any two neighboring points (let's call these nodes) are connected by a resistance R . The following boundary conditions are maintained.

- (i) Voltage on all nodes on the extreme left is maintained at V_0 ;
- (ii) Voltage at the right edge is 0;
- (iii) No current flows out of the top and bottom edges.

Find the voltage at each of the nodes.

Write the problem in terms of a set of algebraic equations and solve those. Choose suitable values of V_0 and R for numerical convenience.

Numerical Methods 2026

Problems on matrices (2)

1. Consider a finite chain of N atoms with one orbital per atom. The ‘tight binding’ Hamiltonian for such a system is written as

$$H = -t \sum_{\langle i,j \rangle, \sigma} (c_{j,\sigma}^\dagger c_{i,\sigma} + c_{i,\sigma}^\dagger c_{j,\sigma}) + \mu \sum_{i,\sigma} n_{i,\sigma}$$

Construct the $N \times N$ Hamiltonian matrix for this system in the basis of the atomic orbitals. Start with a ‘reasonable large’ value of N . Such qualifications are always context-dependent in numerics. But you may choose $N = 10$ to start with.

Find the eigenvalues and eigenvectors.

Analyze how the eigenvalues are distributed.

Comment on the nature of the eigenvectors: are the states localized or delocalized?

Increase N to whatever your compute resources allow. Answer the same questions as above.

Now impose periodic boundary conditions, and answer the same questions.

Do the same exercise in 2D. Note, if you have a $n \times n$ lattice in 2D, there will be $N = n^2$ sites and your Hamiltonian will be $n^2 \times n^2$.

2. Consider the following Hamiltonian for an enharmonic oscillator.

$$H = KE + \frac{1}{2}m\omega^2 x^2 + \epsilon \hbar \omega \left(\frac{m\omega}{\hbar}\right)^2 x^4.$$

Evaluate the matrix elements of the anharmonic term between eigenstates of the harmonic oscillator. You should find that these are non-zero only between states whose quantum numbers differ $\Delta n = 0, 2$ or 4 . The matrix elements are most conveniently written in terms of the integer $m = \min(i, j) - 1$ where the unperturbed states are designated by $n = i - 1$ ($i = 0, 1, 2, \dots$).

Now find the approximate value of the ground state energy of this anharmonic oscillatory solving the problem in the basis of the above three states. Explain why this is justified. Use $\epsilon = 0.1$.